Cognitive Robotics Lab

Lab 8: Create a map data structure with details

Srikar Ganesh 22BAI1080

29-09-2025

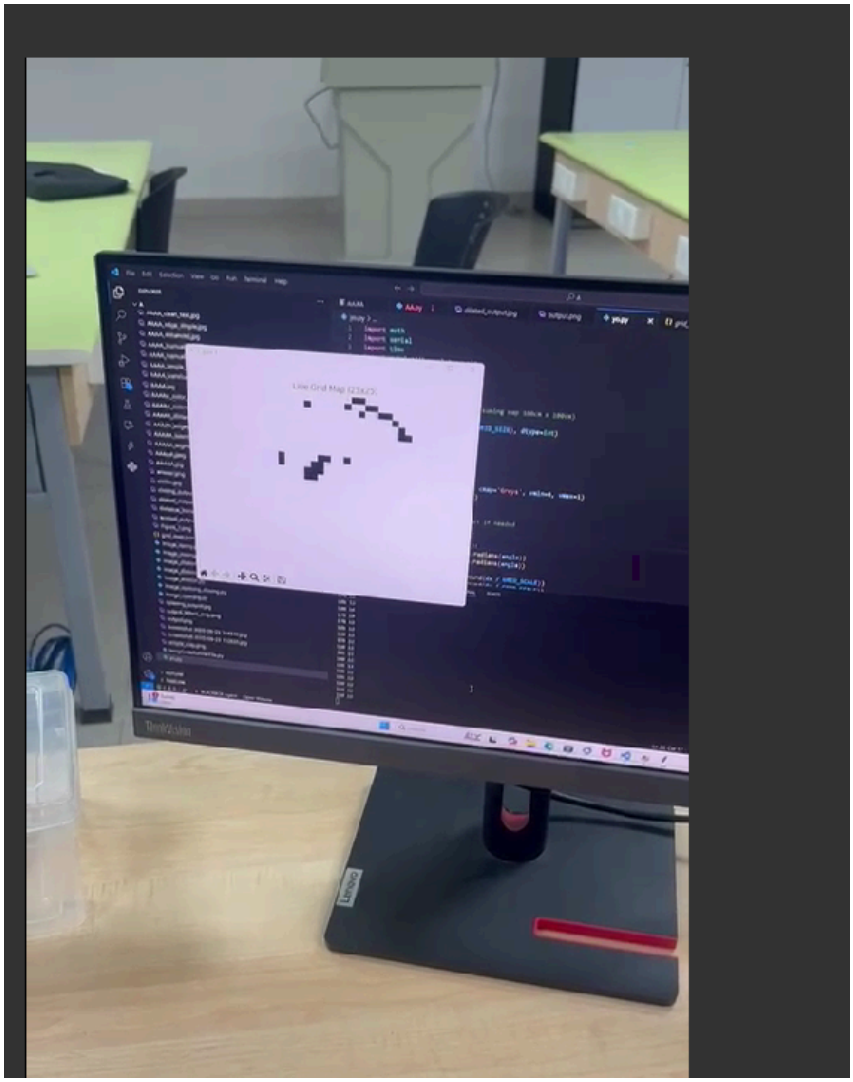
Aim:

To create a simple 2D mapping system using a servo-mounted ultrasonic sensor that scans the surrounding area. The system measures distances at various angles, sends the data via serial communication to a computer, and visualizes obstacles in real-time on a 2D map.

Components:

1. **Arduino Board** (e.g., Arduino Uno) – Controls the servo and reads sensor data.
2. **Servo Motor** – Rotates the ultrasonic sensor to scan angles from 0° to 180°.
3. **Ultrasonic Sensor (HC-SR04)** – Measures distances to obstacles in front of it.
4. **Jumper Wires** – Connect Arduino, servo, and ultrasonic sensor.
5. **Power Supply** – USB or external power for Arduino and servo (if high torque needed).
6. **Computer with Python** – Receives serial data and visualizes the 2D map.
7. **Optional: Breadboard** – For easy connections.

Output:



Source Code:

```
#include <Servo.h>

Servo servo;

const int servoPin = 3;          // Servo control pin
const int trigPin = A5;          // Ultrasonic trigger pin
const int echoPin = A4;          // Ultrasonic echo pin

const int servoMinAngle = 0;     // Minimum servo angle
const int servoMaxAngle = 180;   // Maximum servo angle
const int step = 5;              // Step size for servo sweep
const unsigned long delayBetweenSteps = 100;  // Delay for servo to settle in ms

void setup() {
  Serial.begin(9600);
```

```
  servo.attach(servoPin);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

long readUltrasonicDistance() {
  // Send 10us pulse to trigger
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read echo time in microseconds
  long duration = pulseIn(echoPin, HIGH, 30000);  // timeout 30ms (max ~5m)

  // Calculate distance in cm (speed of sound = 343 m/s)
  long distanceCm = duration * 0.0343 / 2;

  if (duration == 0) {
    return -1;  // no echo received
  } else {
    return distanceCm;
  }
}

void loop() {
  // Sweep servo from 0 to 180 and back
  for (int angle = servoMinAngle; angle <= servoMaxAngle; angle += step) {
    servo.write(angle);
    delay(delayBetweenSteps);

    long distance = readUltrasonicDistance();

    // Send angle and distance as CSV string
    Serial.print(angle);
    Serial.print(",");
    Serial.println(distance);
  }

  for (int angle = servoMaxAngle; angle >= servoMinAngle; angle -= step) {
    servo.write(angle);
    delay(delayBetweenSteps);

    long distance = readUltrasonicDistance();
```

```
    Serial.print(angle);
    Serial.print(",");
    Serial.println(distance);
  }
}
import math
import serial
import time
import matplotlib.pyplot as plt
import numpy as np
import json

GRID_SIZE = 25  # 25×25 grid
GRID_SCALE = 4  # cm per cell (assuming map 100cm x 100cm)

grid_map = np.zeros((GRID_SIZE, GRID_SIZE), dtype=int)

robot_grid_x = GRID_SIZE // 2
robot_grid_y = GRID_SIZE // 2

plt.ion()
fig, ax = plt.subplots()
img_display = ax.imshow(grid_map, cmap='Greys', vmin=0, vmax=1)
plt.title("Live Grid Map (25×25)")
plt.axis('off')

SERIAL_PORT = 'COM8'       # Adjust if needed
BAUD_RATE = 9600

def mark_obstacle(distance, angle):
    dx = distance * math.cos(math.radians(angle))
    dy = distance * math.sin(math.radians(angle))

    grid_x = robot_grid_x + int(round(dx / GRID_SCALE))
    grid_y = robot_grid_y - int(round(dy / GRID_SCALE))

    if 0 <= grid_x < GRID_SIZE and 0 <= grid_y < GRID_SIZE:
        grid_map[grid_y, grid_x] = 1

try:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)
    print(f"Connected to {SERIAL_PORT} at {BAUD_RATE} baud.\n")

    sweep_count = 0
```

```
    prev_angle = None

    while True:
        line = ser.readline().decode('utf-8').strip()
        if line:
            try:
                angle_str, dis_str = line.split(',')
                angle = int(angle_str.strip())
                distance = int(dis_str.strip())
                print(angle, distance)

                # Detect sweep completion: angle wraps around from high to low (e.g., from > 350 to <
10)
                if prev_angle is not None and prev_angle > 350 and angle < 10:
                    sweep_count += 1
                    print(f"Sweep #{sweep_count} completed.")

                    if sweep_count >= 10:
                        print("Completed 10 sweeps. Stopping.")
                        break

                prev_angle = angle

                if distance != -1 and 2 < distance < 100:
                    mark_obstacle(distance, angle)

                img_display.set_data(grid_map)
                plt.draw()
                plt.pause(0.001)

            except ValueError:
                print(f"Invalid line: {line}")

except serial.SerialException as e:
    print(f"Error opening serial port: {e}")
except KeyboardInterrupt:
    print("Interrupted by user. Exiting...")
finally:
    if 'ser' in locals() and ser.is_open:
        ser.close()
        print("Serial port closed.")

# Save grid map as JSON
with open("grid_map.json", "w") as f:
    json.dump(grid_map.tolist(), f)
print("Grid map saved as grid_map.json")
```

Result:

The setup successfully scanned a 180° field in front of the robot, detecting obstacles and plotting them in real-time on a 2D map. The generated map shows positions of walls or objects relative to the robot's starting point, providing a clear spatial layout. This demonstrates a basic but effective method for environmental mapping using low-cost components and can serve as a foundation for autonomous navigation or obstacle avoidance applications.